

Reviewer Pack — Free Entropy Inverse Proportionality to Disjointness Communi...

Entry e9bfefbfd0c1 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-09 00:51:06 UTC

Free Entropy Inverse Proportionality to Disjointness Communication Complexity

Entry ID: e9bfefbfd0c1 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-09 00:51:06 UTC

1. Conjecture

Field A (mathematical branch): Free Probability **Field B** (complexity object): Communication Complexity of DISJOINTNESS

Statement:

For every $n \geq 1$, the free entropy $\Phi(M_n)$ of the communication matrix M_n for the DISJOINTNESS problem satisfies $\Phi(M_n) \geq c / (n \log n)$, where $c > 0$ is a universal constant. Equality holds if and only if M_n is a tensor product of n -dimensional projections.

Rationale (proposer's reasoning):

Free entropy quantifies the non-commutative randomness of a matrix. DISJOINTNESS's $\Omega(n)$ communication lower bound suggests strong non-commutative structure. Free entropy's inverse scaling with n mirrors the information-theoretic cost of coordination in distributed computation.

Taxonomy category: DISPERSION_DISCREPANCY (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 672841dc47438257

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.95	SAFE	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
ALGEBRIZATION	UNCERTAIN	0.80	UNCERTAIN	HITS
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    n = 10 # Default value for n, can be changed in main loop
    c = 1.0 / (n * math.log(n)) # Universal constant c

    def generate_disjointness_matrix(n):
        M = [[0] * n for _ in range(n)]
        for i in range(n):
            M[i][i] = 1
        return M

    def matrix_multiplication(A, B):
        result = [[sum(a*b for a, b in zip(row_a, col_b)) for col_b in zip(*B)] for row_a in A]
        return result

    def gaussian_elimination(A):
        n = len(A)
        for i in range(n):
            # Find the pivot
            max_row = max(range(i, n), key=lambda r: abs(A[r][i]))
            A[i], A[max_row] = A[max_row], A[i]

            # Eliminate below
            for j in range(i + 1, n):
                factor = A[j][i] / A[i][i]
                A[j] = [A[j][k] - factor * A[i][k] for k in range(n)]

        # Back-substitute to find the inverse
        inv_A = [[0] * n for _ in range(n)]
        for i in range(n):
            inv_A[i][i] = 1 / A[i][i]
            for j in range(i + 1, n):
                factor = A[j][i] / A[i][i]
                for k in range(n):
                    inv_A[j][k] -= factor * inv_A[i][k]

        # Normalize the rows
```

```

    for i in range(n):
        norm = sum(inv_A[i])
        inv_A[i] = [x / norm for x in inv_A[i]]

    return inv_A

def eigenvalues(A):
    n = len(A)
    if n == 1:
        return [A[0][0]]

    # Reduce to tridiagonal form
    T = [[0] * n for _ in range(n)]
    Q = [[0] * n for _ in range(n)]
    Q[0][0], Q[n-1][n-1] = 1, 1

    for k in range(2, n):
        h = sum(A[i][i+k-1]**2 for i in range(k-1))
        if h == 0:
            continue
        g = A[k-1][k-1]
        t = (A[k-1][k] * A[k][k-1]) / h
        c = 1 / math.sqrt(1 + t**2)
        s = t * c
        T[:k-1][:k-1] = matrix_multiplication(gaussian_elimination([[c, -s], [s, c]]), A[:k-1])
        T[k-1:k][:k-1] = [[0] * (k-1)]
        T[:k-1][k-1:] = [[0] * (k-1)]
        T[k-1][k-1], T[k][k] = c, -s
        A[:k-1][:k-1] = matrix_multiplication(A[:k-1][:k-1], gaussian_elimination([[c, s], [-s, c]]))

    # Compute eigenvalues of tridiagonal matrix
    eigs = [T[i][i] for i in range(n)]
    return eigs

def free_entropy(eigenvals):
    rho = sum(math.exp(-x) for x in eigenvals)
    entropy = -sum(math.log(rho) * math.exp(-x) / rho for x in eigenvals)
    return entropy

M_n = generate_disjointness_matrix(n)
X = [(M_n[i][j] + 1) / 2 for j in range(n)] for i in range(n)
eigs = eigenvalues(X)
Phi = free_entropy(eigs)

result = {
    "metric_name": "free_entropy",
    "metric_value": Phi,
    "instances_tested": 1,
    "conjecture_holds": Phi >= c,
    "counterexample": "" if Phi >= c else f"Phi={Phi}, expected>=c"
}

return result

if __name__ == "__main__":
    import sys

```

```

seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

results = []
for seed in seeds:
    random.seed(seed)
    result = run_trial(seed)
    print(f"TRIAL: {result}")
    results.append(result)

mean_value = sum(r["metric_value"] for r in results) / len(results)
std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results) / len(results))
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
elif support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
else:
    first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
    counterexample = next(r["counterexample"] for r in results if not r["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e19bcb97.py", line 115, in <module>
    result = run_trial(seed)
              ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e19bcb97.py", line 95, in run_trial
    eigs = eigenvalues(X)
           ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e19bcb97.py", line 71, in eigenvalues
    h = sum(A[i][i+k-1]**2 for i in range(k-1))
        ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e19bcb97.py", line 71, in <genexpr>
    h = sum(A[i][i+k-1]**2 for i in range(k-1))
           ~~~~~^
IndexError: list index out of range

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed with `IndexError`, preventing data collection to evaluate conjecture | next:
Debug the index error in eigenvalues calculation by adding boundary checks for matrix dimensions

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	85696
2	preregistration	ollama_remote	qwen3:8b	0	0	23913
3	novelty	ollama_remote	qwen3:8b	0	0	20514
4	novelty	ollama_remote	qwen3:8b	0	0	12670
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18661
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14700
7	judge	ollama_remote	qwen3:8b	0	0	17618

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 193773 ms total latency. Provider mix: {'ollama_remote': 7}

(full prompt+response transcripts available in `research/audit/e9bfefbfd0c1.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/e9bfefbfd0c1.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/e9bfefbfd0c1.tar.gz` (if generated)